

For creating event emitting example:

For `app.module.ts`:

```
import { Module } from '@nestjs/common';
import { EventEmitterModule } from '@nestjs/event-emitter';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UsersModule } from './users/users.module';

@Module({
  imports: [UsersModule, EventEmitterModule.forRoot()],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Then for `users.service.ts`:

```
import { Injectable } from '@nestjs/common';
import { EventEmitter2 } from '@nestjs/event-emitter';

@Injectable()
export class UsersService {

  constructor(private emitter: EventEmitter2) {}

  createUser(name: string, email: string) {
    // Simulate user creation
    const newUser = { id: Date.now(), name, email };

    // Emit 'user.created' event
    this.emitter.emit('user.created', newUser);

    return newUser;
  }
}
```

Then for `users-created.listener.ts`:

```
import { Injectable, Logger } from '@nestjs/common';
import { OnEvent } from '@nestjs/event-emitter';

@Injectable()
export class UserCreatedListener {
  private readonly logger = new Logger(UserCreatedListener.name);

  @OnEvent('user.created')
  handleUserCreated(user: any) {
    // This runs asynchronously by default
    this.logger.log(`New user created: ${user.name} <${user.email}>`);

    // Example: Send welcome email, update analytics, etc.
  }
}
```

Then for `users.controller.ts`:

```
import { Body, Controller, Post } from '@nestjs/common';
import { UsersService } from './users.service';

@Controller('users')
export class UsersController {
  constructor(private userService: UsersService) {}

  @Post()
  createUser(@Body() createUserDto: { name: string; email: string }) {
    return this.userService.createUser(createUserDto.name, createUserDto.email);
  }
}
```

Then for `users.module.ts`:

```
import { Module } from '@nestjs/common';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { UserCreatedListener } from './user-create.listener';

@Module({
  providers: [UsersService, UserCreatedListener],
  controllers: [UsersController]
})
export class UsersModule {}
```

To create a Nest application instance, we use the core `NestFactory` class. `NestFactory` exposes a few static methods that allow creating an application instance. The `create()` method returns an application object, which fulfills the `INestApplication` interface.

By default, if any error happens while creating the application your app will exit with the code 1. If you want to make it throw an error instead disable the option `abortOnError` (e.g., `NestFactory.create(AppModule, { abortOnError: false })`).

<code>platform-express</code>	Express is a well-known minimalist web framework for node. It's a battle tested, production-ready library with lots of resources implemented by the community. The <code>@nestjs/platform-express</code> package is used by default. Many users are well served with Express, and need take no action to enable it.
<code>platform-fastify</code>	Fastify is a high performance and low overhead framework highly focused on providing maximum efficiency and speed. Read how to use it here .

Whichever platform is used, it exposes its own application interface. These are seen respectively as `NestExpressApplication` and `NestFastifyApplication`.

When you pass a type to the `NestFactory.create()` method, as in the example below, the `app` object will have methods available exclusively for that specific platform. Note, however, you don't **need** to specify a type **unless** you actually want to access the underlying platform API.

```
const app = await NestFactory.create<NestExpressApplication>(AppModule);
```



To speed up the development process (x20 times faster builds), you can use the [SWC builder](#) by passing the `-b swc` flag to the `start` script, as follows `npm run start -- -b swc`.

```

C:\Tests\j-l-nestjs\jloka-test>npm run start -- -b swc

> jloka-test@0.0.1 start
> nest start -b swc

Error Failed to load "@swc/cli" and/or "@swc/core" required packages. Please, make sure to install them as development dependencies.

C:\Tests\j-l-nestjs\jloka-test>npm install -D @swc/cli @swc/core

added 85 packages, and audited 807 packages in 2m

163 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Tests\j-l-nestjs\jloka-test>
C:\Tests\j-l-nestjs\jloka-test>
C:\Tests\j-l-nestjs\jloka-test>
C:\Tests\j-l-nestjs\jloka-test>npm run start -- -b swc

> jloka-test@0.0.1 start
> nest start -b swc

> SWC Running...
Successfully compiled: 11 files with swc (263.41ms)
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [NestFactory] Starting Nest application...
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] UserCacheModule dependencies initialized +45ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] ConfigHostModule dependencies initialized +1ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] DiscoveryModule dependencies initialized +1ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] AppModule dependencies initialized +2ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] ConfigModule dependencies initialized +2ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] EventEmitterModule dependencies initialized +2ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] UsersModule dependencies initialized +3ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [InstanceLoader] CacheModule dependencies initialized +2ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [RoutesResolver] AppController {/}: +9ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [RouterExplorer] Mapped {/, GET} route +7ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [RoutesResolver] UsersController {/users}: +1ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [RouterExplorer] Mapped {/users, POST} route +3ms
[Nest] 13332 - 12/14/2025, 4:12:20 PM LOG [NestApplication] Nest application successfully started +11ms

```

```

C:\Tests\j-l-nestjs\jloka-test>nest g resource simple
✓ What transport layer do you use? REST API
✓ Would you like to generate CRUD entry points? Yes
CREATE src/simple/simple.controller.ts (959 bytes)
CREATE src/simple/simple.controller.spec.ts (596 bytes)
CREATE src/simple/simple.module.ts (264 bytes)
CREATE src/simple/simple.service.ts (661 bytes)
CREATE src/simple/simple.service.spec.ts (478 bytes)
CREATE src/simple/dto/create-simple.dto.ts (33 bytes)
CREATE src/simple/dto/update-simple.dto.ts (181 bytes)
CREATE src/simple/entities/simple.entity.ts (24 bytes)
UPDATE package.json (2395 bytes)
UPDATE src/app.module.ts (567 bytes)
✓ Packages installed successfully.

```

Controllers are responsible for **handling** incoming **requests** and **sending responses** back to the client.

The concept that Nest uses **two different options for manipulating responses**:

Standard (recommended)	Using this built-in method, when a request handler returns a JavaScript object or array, it will automatically be serialized to JSON. When it returns a JavaScript primitive type (e.g., <code>string</code>, <code>number</code>, <code>boolean</code>), however, Nest will send just the value without attempting to serialize it. This makes response handling simple: just return the value, and Nest takes care of the rest. Furthermore, the response's status code is always 200 by default, except for POST requests which use 201. We can easily change this behavior by adding the <code>@HttpCode(...)</code> decorator at a handler-level (see Status codes).
Library-specific	We can use the library-specific (e.g., Express) response object, which can be injected using the <code>@Res()</code> decorator in the method handler signature (e.g., <code>findAll(@Res() response)</code>). With this approach, you have the ability to use the native response handling methods exposed by that object. For example, with Express, you can construct responses using code like <code>response.status(200).send()</code>.

WARNING

Nest detects when the handler is using either `@Res()` or `@Next()`, indicating you have chosen the library-specific option. If both approaches are used at the same time, the Standard approach is **automatically disabled** for this single route and will no longer work as expected. To use both approaches at the same time (for example, by injecting the response object to only set cookies/headers but still leave the rest to the framework), you must set the `passthrough` option to `true` in the `@Res({ passthrough: true })` decorator.

Request object

Handlers often need access to the client's **request** details. Nest provides access to the **request object** from the underlying platform (Express by default). You can access the request object by instructing Nest to inject it using the `@Req()` decorator in the handler's signature.

cats.controller.ts

JS

```
import { Controller, Get, Req } from '@nestjs/common';
import type { Request } from 'express';

@Controller('cats')
export class CatsController {
  @Get()
  findAll(@Req() request: Request): string {
    return 'This action returns all cats';
  }
}
```

<code>@Request(), @Req()</code>	<code>req</code>
<code>@Response(), @Res() *</code>	<code>res</code>
<code>@Next()</code>	<code>next</code>
<code>@Session()</code>	<code>req.session</code>
<code>@Param(key?: string)</code>	<code>req.params</code> / <code>req.params[key]</code>
<code>@Body(key?: string)</code>	<code>req.body</code> / <code>req.body[key]</code>
<code>@Query(key?: string)</code>	<code>req.query</code> / <code>req.query[key]</code>

<code>@Headers(name?: string)</code>	<code>req.headers / req.headers[name]</code>
<code>@Ip()</code>	<code>req.ip</code>
<code>@HostParam()</code>	<code>req.hosts</code>

`@Res()` is simply an alias for `@Response()`. Both directly expose the underlying native platform `response` object interface. When using them, you should also import the typing's for the underlying library (e.g., `@types/express`) to take full advantage. Note that when you inject either `@Res()` or `@Response()` in a method handler, you put Nest into **Library-specific mode** for that handler, and you become responsible for managing the response. When doing so, you must issue some kind of response by making a call on the `response` object (e.g., `res.json(...)` or `res.send(...)`), or the HTTP server will hang.

```
import { Body, Controller, Get, Post } from '@nestjs/common';
import { UsersService } from './users.service';

@Controller('users')
export class UsersController {
  constructor(private userService: UsersService) {}

  @Post()
  createUser(@Body() createUserDto: { name: string; email: string }) {
    return this.userService.createUser(createUserDto.name, createUserDto.email);
  }

  // in this way we can use supported way to paths: *, by adding *path where path
  // is the prefix
  // here the path is $
  @Get('/loka/*$')
  getLoka() {
    return "My name is: Jafar Loka";
  }
}
```

This approach works on both `Express` and `Fastify`. However, with the latest release of Express (v5), the routing system has become more strict. In pure Express, you must use a named wildcard to make the route work—for

example, `abcd/*splat`, where `splat` is simply the name of the wildcard parameter and has no special meaning. You can name it anything you like. That said, since Nest provides a compatibility layer for Express, you can still use the asterisk (`*`) as a wildcard.

When it comes to asterisks used in the **middle of a route**, Express requires named wildcards (e.g., `ab{*splat}cd`), while Fastify does not support them at all.

Response headers

To specify a custom response header, you can either use a `@Header()` decorator or a library-specific response object (and call `res.header()` directly).

```
@Post()
@Header('Cache-Control', 'no-store')
create() {
  return 'This action adds a new cat';
}
```

HINT

Import `Header` from the `@nestjs/common` package.

Redirection

To redirect a response to a specific URL, you can either use a `@Redirect()` decorator or a library-specific response object (and call `res.redirect()` directly).

`@Redirect()` takes two arguments, `url` and `statusCode`, both are optional. The default value of `statusCode` is `302` (`Found`) if omitted.

```
@Get()
@Redirect('https://nestjs.com', 301)
```

HINT

Sometimes you may want to determine the HTTP status code or the redirect URL dynamically. Do this by returning an object following the `HttpRedirectResponse` interface (from `@nestjs/common`).

Returned values will override any arguments passed to the `@Redirect()` decorator. For example:

```
@Get('docs')
@Redirect('https://docs.nestjs.com', 302)
getDocs(@Query('version') version) {
  if (version && version === '5') {
    return { url: 'https://docs.nestjs.com/v5/' };
  }
}
```

```
@Get(':id')
findOne(@Param() params: any): string {
  console.log(params.id);
  return `This action returns a #${params.id} cat`;
}
```

The `@Param()` decorator is used to decorate a method parameter (in the example above, `params`), making the **route** parameters accessible as properties of that decorated method parameter inside the method. As shown in the code, you can access the `id` parameter by referencing `params.id`. Alternatively, you can pass a specific parameter token to the decorator and directly reference the route parameter by name within the method body.

Sub-domain routing

The `@Controller` decorator can take a `host` option to require that the HTTP host of the incoming requests matches some specific value.

```
@Controller({ host: 'admin.example.com' })
export class AdminController {
  @Get()
  index(): string {
    return 'Admin page';
  }
}
```

WARNING

Since **Fastify** does not support nested routers, if you are using sub-domain routing, it is recommended to use the default Express adapter instead.

Similar to a route `path`, the `host` option can use tokens to capture the dynamic value at that position in the host name. The host parameter token in the `@Controller()` decorator example below demonstrates this usage. Host parameters declared in this way can be accessed using the `@HostParam()` decorator, which should be added to the method signature.

```
@Controller({ host: ':account.example.com' })
export class AccountController {
  @Get()
  getInfo(@HostParam('account') account: string) {
    return account;
  }
}
```

Nest, nearly everything is shared across incoming requests. This includes resources like the [database connection pool](#), [singleton services](#) with [global state](#), and [more](#). It's important to understand that Node.js doesn't use the [request/response Multi-Threaded Stateless Model](#), where each request is handled by a separate thread. As a result, using singleton instances in Nest is completely **safe** for our applications.

There are specific edge cases where having request-based lifetimes for controllers may be necessary. Examples include [per-request caching](#) in GraphQL applications, [request tracking](#), or [implementing multi-tenancy](#).
